INDEX

Name : Animesh Yadav Subject :

Std : Sec. : OS (Lab) Roll No. : 049

Sl No.	Date	Title	Marks	Teacher's Sign/Remarks
1)	24/02/026	To find second smallest element in an array		
2)	24/02/026	To find sum of left diagonals of a matrix		
3)	3/03/026	To simulate CPU scheduling algorithm (FCFS & SJF)		
4)	10/03/026	To simulate CPU scheduling algorithm to find TAT and waiting Time for given priority (Pre-emptive & non)		
5)	17/03/026	Round Robin		
6)	24/03/026	multilevel queue		
7)	31/03/026	EDF & RMS	10	
8)	21/4/026	Producer consumer		
9)	21/4/026	Dining Philosopher		
10)	28/4/026	Deadlock (Safety)		
11)	5/5/026	Banker's Algorithm		
12)	5/5/026	Deadlock Detection		
13)	12/5/026	memory Allocation		
14)	26/5/026	Replacement Algorithm		
			10	

WEEK - 1

#) To find second smallest element in an array.

```
#include <stdio.h>
int main() {
    int n, i, smallest, secsmallest;
    Printf("Enter the number of elements in the array: ");
    scanf("%d", &n);

    if (n<2) {
    Printf("Array must have at least 2 elements.\n");
    return 1;
    }
    int arr[n];
    Printf("Enter %d elements:\n", n);
    for (i=0; i<n; i++) {
    scanf("%d", &arr[i]);
    }
    if (arr[0]<arr[1]) {
     smallest = arr[0];
     secsmallest = arr[0];
    }
     for (i=2; i<n; i++) {
      if (arr[i]<smallest) {
      secsmallest = smallest;
      smallest = arr[i];
      }
    else if (arr[i]<secsmallest && arr[i] != smallest) {
        secsmallest = arr[i];
      }
    }
```

```
    if (smallest == secsmallest) {
    printf("there is no second smallest element (all elements are
                equal).\n");
    }
    else {
    printf("the second smallest element is: %d\n", secsmallest);
    }
    return 0;
}
```

Output:

Enter the number of elements in the array: 1
Array must have as least 2 elements.

Enter number of elements in the array: 3
Enter 3 elements: 2
2
2
there is no second smallest elements/all elements are equal)

Enter number of elements in the array: 4
Enter 4 elements: 3
2
1
4
The second smallest element is 2

#) To find the sum of the left diagonals of a matrix.

```
#include <stdio.h>
int main() {
int n, i, j, sum=0;
printf("Enter the size of the square matrix: ");
scanf("%d", &n);

int matrix[n][n];

printf("enter %d x %d matrix elements : \n", n, n);
for(i=0; i<n; i++) {
for(j=0; j<n.; j++) {
scanf("%d", &matrix[i][j]);
}
}

for(i=0; i<n; i++) {
sum+= matrix[i][i];
}
printf("sum of left diagonal (primary diagonal) elements: %d\n", sum);
return 0;
}
```

Output:

Enter the size of the square matrix: 2
enter 2x2 matrix elements:
3
5
2
7
Sum of left diagonal (primary diagonal) elements: 10

week-2

To stimulate CPU scheduling algorithm

1) FCFS
2) SJF

code

```
#include<stdio.h>
#define MAX 20
void fcfs (int n, int at[], int bt[]){
int wt[MAX], tat[MAX];
int time = 0;
float total_wt = 0, total_tat = 0;

for (int i = 0; i<n; i++){
if (time < at[i]){
time = at[i];} rt[i] = time - at[i];
wt[i] = time - at[i]; wt[i] = rt[i];
time += bt[i]; ct[i] = time;
tat[i] = ct[i] - at[i];

total_wt += wt[i];
total_tat += tat[i]; total_rt += rt[i];
}
printf("\n --- FCFS --- \n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\tRT\n");
for (int i = 0; i<n; i++){
printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
      i+1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);

}
printf("\n Average waiting time = %.2f", total_wt/n);
printf("\n Average turn around time = %.2f", total_tat/n);
printf("\n Average Response time = %.2f\n", total_rt/n);

}
```

```
void SJF_non_preemptive(int n, int at[], int bt[]){
int wt[MAX]={0}, tat[MAX]={0}, ct[MAX]={0}, rt[MAX]={0};
int completed[MAX]={0};
int time=0, count=0;
float total_wt=0, total_tat=0, total_rt=0;

while(count<n){
int min=9999, idx=-1;
for(int i=0; i<n; i++){
if(at[i]<=time && !completed[i])
{
if(bt[i]<min){
min=bt[i];
idx=i;
}
}
}
if(idx!=-1){
rt[idx]=time-at[idx];
wt[idx]=rt[idx];
time+=bt[idx];
ct[idx]=time;
tat[idx]=ct[idx]-at[idx];

total_wt+=wt[idx];
total_tat+=tat[idx];
total_rt+=rt[idx];

completed[idx]=1;
count++;
}
else{
time++;
}
}
printf("\n--- SJF Non-Preemptive ---\n");
printf("P\tAT\tBT\tCT\tTAT\tWT\tRT\n");
for(int i=0; i<n; i++){
```

```
printf("%d\t%d\t%d\t%d\t%d\t%d\n", i+1, at[i], bt[i], ct[i],
        tat[i], wt[i], rt[i]);
}
printf("\n Average waiting time = %.2f", total_wt/n);
printf("\n Average Turnaround time = %.2f", total_tat/n);
printf("\n Average Response time = %.2f", total_rt/n);
}
void sjf_preemptive(int n, int at[], int bt[]){
int rt_rem[MAX], wt[MAX]={0}, tat[MAX]={0},
int ct[MAX]={0}, rt[MAX];
int first_start[MAX];
for(int i=0; i<n; i++){
rt_rem[i] = bt[i];
first_start[i]=-1;
}
int complete=0, time=0;
float total_wt=0, total_tat=0, total_rt=0;
while(complete<n){
int min=9999, idx=-1;
for(int i=0; i<n; i++){
if(at[i]<=time && rt_rem[i]>0)
{
if(rt_rem[i]<min){
min=rt_rem[i];
idx=i;
}
}
}
if(idx==-1){
time++;
continue;
}
```

```
if (first_start[idx] == -1)
    first_start[idx] = time;
rt_rem[idx]--;
time++;
if (rt_rem[idx] == 0) {
    completed++;
    ct[idx] = time;
    tat[idx] = ct[idx] - at[idx];
    wt[idx] = tat[idx] - bt[idx];
    rt[idx] = first_start[idx] - at[idx];
    total_wt += wt[idx];
    total_tat += tat[idx];
    total_rt += rt[idx];
}
}
printf("\n--- SJF Preemptive (SRTF) ---\n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\tRT\n");
for (int i=0; i<n; i++){
printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", i+1, at[i], bt[i], ct[i],
        tat[i], wt[i], rt[i]);
}
printf("\nAverage waiting time = %.2f", total_wt/n);
printf("\nAverage turnaround time = %.2f", total_tat/n);
printf("\nAverage response time = %.2f\n", total_rt/n);
}
int main(){
int n, choice, at[MAX], bt[MAX];
printf("Enter number of Process:");
scanf("%d", &n);
for (int i=0; i<n; i++) {
printf("Enter number of Processes:");
scanf("%d", &n);
for (int i=0; i<n; i++) {
```

```
scanf("%d", &at[i]);
printf("Burst time:");
scanf("%d", &bt[i]);
printf("\n choose algorithm:");
printf("\n1. FCFS :\n2. SJF non-preemptive \n3. SJF preemptive(SRTF)");
printf("\n enter choice:");
scanf("%d", &choice);

switch(choice){

case1:
    FCFS(n, at, bt);
    break;

case2: SJF_ non preemptive (n, at, bt);
    break;

case3: SJF_ preemptive (n, at, bt);
    break;

default:
  printf("Invalid choice:");
}
return 0;
}
```

~~output~~:

```
Enter number of processes: 4

Process: 1
Arrival time: 0
Burst time: 5

Process: 2
Arrival time: 4
Burst time: 7
```

Process 3
Arrival time: 3
Burst time: 3

Process 4
Arrival time: 6
Burst time: 9

choose algorithm:
1) FCFS
2) SJF non-preemptive
3) SJF preemptive (SRTF)

Enter choice: 1

---- FCFS ----

PID	AT	BT	CT	TAT	WT	RT
1	0	5	5	5	0	0
2	4	7	12	8	1	1
3	3	3	15	12	9	9
4	6	9	24	18	9	9

Average waiting time = 4.75
Average turnaround time = 10.75
Average response time = 4.75

Enter choice: 2

---- SJF non-preemptive ----

PID	AT	BT	CT	TAT	WT	RT
1	0	5	5	5	0	0
2	4	7	15	11	4	4
3	3	3	8	5	2	2
4	6	9	24	18	9	9

Average waiting time = 3.75

Average turnaround time = 9.75

Average response time = 3.75

Enter choice : 3

- - - - - SJF Preemptive - - - -

PID	AT	BT	CT	TAT	WT	RT
1	0	5	5	5	0	0
2	4	7	15	11	4	4
3	3	3	8	5	2	2
4	6	9	24	18	9	9

Average waiting time = 3.75

Average turnaround time = 9.75

Average response time = 3.75

Comparison:

SJF Preemptive is the best, then SJF Non-Preemptive and then FCFS is slowest.

week-3

#To stimulate the following CPU scheduling algorithm to find turn around time and waiting time.

→ Priority (Pre-emptive & non pre-emptive)

```
#include<stdio.h>
#define max 20
void priority_np(int n, int at[], int bt[], int pr[])
{
  int wt[max]={0}, tat[max], st[max], ct[max], done[max];
  int order[max], start[max], end[max];
  int time=0, count=0, k=0;
  while(count<n){
  int idx=-1, min=9999;
  for(int i=0; i<n; i++)
  if(at[i]<=time && !done[i] && pr[i]<min)
  {
   min=pr[i];
   idx=i;
  }
  if(idx!=-1){
  start[k]=time;
  order[k]=idx;
  tat[idx]=time-at[idx];
  wt[idx]=time-at[idx];
  time+=bt[idx];
  end[k]=time;
  ct[idx]=time;
  tat[idx]=ct[idx]-at[idx];
  done[idx]=1;
  count++;
  k++;
}
```

```
else
time++;
}
printf("\n Gantt chart: \n|");
for(int i=0; i<k; i++)
printf("P%d|", order[i]+1);
printf("\n%d", start[0]);
for(int i=0; i<k; i++)
printf("%d", end[i]);
printf("\n\nProcess\tArrival\tBurst\tWaiting\tTurnaround\n");
for(int i=0; i<n; i++)
printf("\nP%d\t%d\t%d\t%d\t%d", i+1, at[i],
    bt[i], pr[i], ct[i], wt[i], tat[i], rt[i]);
}

void priority_p(int n, int at[], int bt[], int pr[]) {
int rt[MAX], wt[MAX]={0}, tat[MAX], st[MAX], ct[MAX], gantt
  [200], g=0;
for (int i=0; i<n; i++) {
 rt_bt[i] = bt[i];
 rt[i] = -1;
}
 int time=0, complete=0;
while (complete<n) {
int idx=-1, min=999;
for (int i=0; i<n; i++) {
if (at[i]<=time && rt_bt[i]>0 && pr[i]<min) {
 min=pr[i];
 idx = i;
 }
}
```

```
if(idx!=-1){
gantt[gtt]=idx;
if(rt[idx]==-1)
rt[idx]=time-at[idx];
rt_bt[idx]--;
time++;

if(rt_bt[idx]==0){
complete++;
ct[idx]=time;
tat[idx]=ct[idx]-at[idx];
wt[idx]=tat[idx]-bt[idx];
}
}
else{
gantt[gtt]=-1;
time++;
}
}
printf("\n Gantt chart:\n|");
for(int i=0;i<g;i++){
if(gantt[i]==-1)
printf("idle|");
else
printf("P%d|",gantt[i]+1);
}
printf("\n0");
for(int i=1;i<=g;i++)
printf("%2d",i);
printf("\n\nPid\tBT\tCT\tTAT\tWT\n");
for(int i=0;i<n;i++)
printf("P%d\t%d\t%d\t%d\t%d\n",i+1,bt[i],ct[i],tat[i],wt[i]);
}
```

```
int main (){
int n, choice;
int at[MAX], bt[MAX], pr[MAX];
Printf("Processess:");
Scanf("%d", &n);

for (int i=0; i<n; i++){
Printf("AT BT PR for P%d:", i+1);
Scanf("%d %d %d", &at[i], &bt[i], &pr[i]);
}
Printf("\n1. Priority non-Preemptive \n2. Priority Preemptive \n
   choice:");
Scanf("%d", &choice);

Switch (choice){
 Case 1: priority_nn(n, at, bt, pr);
 break;

Case 2: priority_p(n, at, bt, pr);
      break;
}
 return 0;
}
```

Output:

Processess: 5

AT BT PR for P1: 0
3
5

AT BT PR for P2: 2
2
3

AT BT PR for P4: 4
4
4

AT BT PR for PS: 6

1
1

1.) Priority Non-Preemptive
2) Priority Preemptive.

Choice: 1

Gantt Chart:

| P1 | P3 | P5 | P2 | P4 |
0 3 8 9 11 15

PID	AT	BT	PR	CT	TAT	WT	RT
1	0	3	5	3	3	0	0
2	2	2	3	11	9	7	7
3	3	5	2	8	5	0	0
4	4	4	4	15	11	7	7
5	6	1	1	9	3	2	2

Choice: 2

10/3/26

Gantt Chart:

| P1 | P1 | P2 | P3 | P3 | P3 | P4 | P3 | P3 | P2 | P4 | P4 | P4 | P4 | P1 |
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

PID	AT	BT	PR	CT	WT	TAT	RT
1	0	3	5	15	12	15	0
2	2	2	3	10	6	8	0
3	3	5	2	9	1	6	0
4	4	4	4	14	6	10	6
5	6	1	1	7	0	1	0

1) Priority non-preemptive:

Averages

Average Turn Around Time: 6.2
Average waiting Time = 3.2
Average Response Time = 3.2

2) Priority Preemptive:

Averages

Average Turn Around Time = 5
Average waiting time = 5
Average Response Time = 1.2

Comparison:

priority non-preemptive is best than priority preemptive because processes run continuously without interuption, which reduces context switching and results in smaller average waiting time and turn around time.

Round Robin

```
#include<stdio.h>
#define MAX 20
void roundRobin(int n, int at[], int bt[], int quantum)
{
int rt_bt[MAX], ct[MAX], wt[MAX], tat[MAX], rt[MAX];
int queue[100], front=0, rear=0;
int visited[MAX]={0};

for(int i=0; i<n; i++)
{
 rt_bt[i]=bt[i];
 rt[i]=-1;
}
int time=0, completed=0;
int gantt_p[100], gantt_t[100], g=0;

queue[rear++]=0;
visited[0]=1;

while(completed<n)
{
 int i=queue[front++];

if(rt[i]==-1)
 rt[i]=time-at[i];

int exec=(rt_bt[i]>quantum)?quantum:rt_bt[i];

gantt_p[g]=i;
gantt_t[g++]=time;

time+=exec;
rt_bt[i]-=exec;

for(int j=0; j<n; j++)
{
 if(at[j]<=time && visited[j]==0)
 {
```

```
    queue [rear++] = j;
    visited [j] = 1;
    }
  }
  if (rt_bt[i] > 0)
  {
   queue [rear++] = i;
  }
  else
  {
   ct[i] = time;
   completed++;
  }
  if (front == rear)
  {
   for(int j = 0; j < n; j++)
   {
    if (visited [j] == 0)
    {
     queue [rear++] = j;
     visited [j] = 1;
     time = at[j];
     break;
    }
   }
  }
 }
 gantt_t [g] = time;
 float total_wt = 0, total_tat = 0, total_rt = 0;
 printf("\nPID:\t AT \t BT \t CT \t TAT \t WT \t RT \n");
 for(int i = 0; i < n; i++)
 {
  tat[i] = ct[i] - at[i];
```

```
wt[i] = tat[i] - bt[i];

total_wt += wt[i];
total_tat += tat[i];
total_rt += rt[i];

printf("%d\t%d\t%d\t%d\t%d\t%d\n",
    i+1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i]); }

printf("\n Average WT = %.2f", total_wt / n);
printf("\n Average TAT = %.2f", total_tat / n);
printf("\n Average RT = %.2f\n", total_rt / n);
printf("\n Gantt Chart:\n");

for (int i=0; i<g; i++)
printf("|%d", gantt_p[i] + 1);
printf("|\n");

for (int i=0; i<=g; i++)
printf("%d ", gantt_t[i]);
printf("\n");
int main()
{
int n, quantum;
int at[MAX], bt[MAX];
printf("Enter number of processes:");
scanf("%d", &n);

for (int i=0; i<n; i++)
{
printf("\n Process %d\n", i+1);
printf("Arrival time:");
scanf("%d", &at[i]);
```

```
        printf("Burst Time: ");
        scanf("%d", &bt[i]);
    }

    printf("\n Enter Time quantum: ");
    scanf("%d", &quantum);

    roundRobin(n, at, bt, quantum);

    return 0;

}
```

O/P:

```
Enter number of processes: 5

Process 1
Arrival Time: 0
Burst Time: 5

Process 2
Arrival time: 1
Burst Time: 3

Process 3
Arrival Time: 2
Burst Time: 1

Process 4
Arrival time: 3
Burst Time: 1

Process 5
Arrival Time: 4
Burst Time: 4

Enter Time Quantum: 2
```

PID	AT	BT	CT	TAT	WT	RT
1	0	5	15	15	10	0
2	1	3	12	11	8	1
3	2	6	19	17	11	2
4	3	1	9	6	5	5
5	4	4	17	13	9	5

Average WT = 8.6

Average TAT = 12.4

Average RT = 2.6

Gantt Chart:

P1	P2	P3	P1	P4	P5	P2	P3	P1	P5	P3
0	2	4	6	8	9	11	12	14	15	17	19

Enter Time quantum: 4

PID	AT	BT	CT	TAT	WT	RT
1	0	5	14	14	9	0
2	1	3	7	6	3	3
3	2	1	8	6	5	5
4	3	1	9	6	5	5
5	4	4	13	9	5	5

19/3/26

Average WT = 5.40

Average TAT = 8.20

Average RT = 3.6

Gantt Chart:

P1	P2	P3	P4	P5	P1
0	4	7	8	9	13	14

Conclusion:

Round Robin scheduling was implemented and analyzed. It ensures fairness by giving each process equal CPU time and avoids starvation. The performance depends on time quantum - small values increase overhead, while large values behave like FCFS.

Multilevel queue scheduling

```
#include <stdio.h>
#include <string.h>
#define MAX 100

typedef struct {
  int pid;
  int arrival;
  int burst;
  int remaining;
  int completion;
  int waiting;
  int turnaround;
  char type[10];
} process;

typedef struct {
  int items[MAX];
  int front, rear;
} queue;

void initqueue(queue *q) {
  q->front = q->rear = -1;
}

int isEmpty(queue *q) {
  return (q->front == -1);
}

void enqueue(queue *q, int val) {
  if (q->rear == MAX-1) return;
  if (q->front == -1) q->front = 0;
  q->items[++q->rear] = val;
}

int dequeue(queue *q) {
  if (isEmpty(q)) return -1;
  int val = q->items[q->front];
  if (q->front == q->rear)
    q->front = q->rear = -1;
  else
    q->front++;
```

```
    return val;
}
void sortByArrival(Process P[], int n) {
  Process temp;
for (int i=0; i<n-1; i++) {
for (int j=i+1; j<n; j++) {
if (P[i].arrival > P[j].arrival) {
  temp = P[i];
  P[i] = P[j];
  P[j] = temp;
   }
  }
 }
}

int main() {
  int n;
  Process P[MAX];
printf("Enter number of processes: ");
scanf("%d", &n);

for(int i=0; i<n; i++) {
 Printf("\nProcess %d\n", i+1);
 Printf("PID: ");
 Scanf("%d", &P[i].Pid);
 Printf("Arrival Time: ");
 Scanf("%d", &P[i].arrival);
 Printf("Burst Time: ");
 Scanf("%d", &P[i].burst);
 Printf("Type (System/User): ");
 Scanf("%s", P[i].type);
 P[i].remaining = P[i].burst;
}
```

```
Sort By Arrival(P,n);

Queue systemQ, userQ;
initQueue(&systemQ);
initQueue(&userQ);

int time=0, completed=0, i=0;
int current=-1;

int gantt[MAX], g_time[MAX];
int g_index=0;

while(completed<n){
while(i<n && P[i].arrival<=time){
if(strcmp(P[i].type,"system")==0)
  enqueue(&systemQ, i);
else
  enqueue(&userQ, i);

i++;
}

if(current!=-1){
if(strcmp(P[current].type,"user")==0 &&
 !isEmpty(&systemQ)){
enqueue(&userQ, current);
current=-1;
 }
}

if(current==-1){
if(!isEmpty(&systemQ))
  current=dequeue(&systemQ);
else if(!isEmpty(&userQ))
 current=dequeue(&userQ);
else{
  time++;
  continue;
 }
}
```

```
gantt[g_index] = p[current].pid;
g_time[g_index] = time;
g_index++;

p[current].remaining--;
time++;

if (p[current].remaining == 0) {
    p[current].completion = time;
    completed++;
    current = -1;
}
}
g_time[g_index] = time;
float totalWT = 0, totalTAT = 0;
for(int i=0; i<n; i++) {
p[i].turnaround = p[i].completion - p[i].arrival;
p[i].waiting = p[i].turnaround - p[i].burst;

totalWT += p[i].waiting;
totalTAT += p[i].turnaround;
}
printf("\n\nGantt Chart:\n|");
for(int i=0; i<g_index; i++) {
printf("P%d|", gantt[i]);
}
printf("\n%d", g_time[0]);
for(int i=1; i<=g_index; i++) {
printf("%2d", g_time[i]);
}
printf("\n\nPid\tArrival\tBurst\tCompletion\tTAT\tWT\n");
for(int i=0; i<n; i++) {
printf("P%d\t%d\t%d\t%d\t\t%d\t%d\n",
    p[i].pid,
```

P[i].type,
P[i].arrival,
P[i].burst,
P[i].completion
P[i].turnaround
P[i].waiting)
}
printf("\nAverage waiting time = %.2f", totalWT/n);
printf("\nAverage turnaround time = %.2f\n", totalTAT/n);
return 0;
}

OUTPUT:

Enter number of process: 4

Process: 1
PID: 0
Arrival time: 0
Burst time: 5
Type(system/user): system

Process: 2
PID: 1
Arrival time: 2
Burst time: 3
Type(system/user): user

Process 3
PID: 2
Arrival time: 1
Burst time: 4
Type(system/user): system

process: 4
ID: 3
Arrival time: 3
Burst time: 2
Type: (system/user): user

Gantt chart:

P_0	P_2	P_1	P_3
0	5	9	12	14

PID	Type	AT	BT	CT	TAT	WT
P_0	system	0	5	5	5	0
P_2	system	1	4	9	8	4
P_1	user	2	3	12	10	7
P_3	user	3	2	14	11	9

Average WT = 5.00

Average TAT = 8.50

24/3/26

EDF & RMS

```
#include <stdio.h>
#define MAX 10

typedef struct {
  int id;
  int execution;
  int period;
  int deadline;
} Task;

void edf(Task t[], int n) {
printf("\n=== Earliest Deadline First (EDF) Scheduling ===\n");
int time = 0, completed = 0;
int remaining[MAX], ct[MAX] = {0}, wt[MAX], tat[MAX];

for (int i = 0; i < n; i++)
 remaining[i] = t[i].execution;

 while (completed < n) {
 int earliest = 9999, selected = -1;

 for (int i = 0; i < n; i++) {
 if (remaining[i] > 0 && t[i].deadline < earliest) {
 earliest = t[i].deadline;
 selected = i;
 }
}
if (selected != -1) {
remaining[selected]--;
time++;

if (remaining[selected] == 0) {
ct[selected] = time;
```

```
tat[selected] = ct[selected];
wt[selected] = tat[selected] - t[selected].execution;
completed ++;
}
} else {
time++;
}
}
printf("ID\tBT\tDeadline\tCT\tWT\tTAT\n");
for(int i=0; i<n; i++) {
printf("%d\t%d\t%d\t%d\t%d\t%d\n",
t[i].id, t[i].execution, t[i].deadline,
ct[i], wt[i], tat[i]);
}
}
void rms(task t[], int n) {
printf("\n==== Rate monotonic scheduling (RMS) ====\n");
int time=0, completed=0;
int remaining[MAX], ct[MAX]={0}, wt[MAX], tat[MAX];
for(int i=0; i<n; i++)
remaining[i] = t[i].execution;
while(completed<n) {
int min_period=9999, selected=-1;
for(int i=0; i<n; i++) {
if(remaining[i]>0 && t[i].period<min_period) {
min_period = t[i].period;
```

```
    selected = i;
  }
}
if (selected != -1) {
  remaining[selected]--;
  time++;
if (remaining[selected] == 0) {
  ct[selected] = time;
  tat[selected] = ct[selected];
  wt[selected] = tat[selected] - t[selected].execution;
  completed++;
  }
} else {
   time++;
  }
}
printf("ID\tBT\tPeriod\tCT\tWT\tTAT\n");
for(int i=0; i<n; i++) {
printf("%d\t%d\t%d\t%d\t%d\t%d\n",
   t[i].id, t[i].execution, t[i].period,
   ct[i], wt[i], tat[i]);
  }
}

int main() {
   int n;
   Task t[MAX];
   printf("Enter number of tasks:");
   scanf("%d", &n);
for(int i=0; i<n; i++) {
```

```
printf("\nTask %d\n", i+1);
t[i].id = i+1;
printf("Burst Time: ");
scanf("%d", &t[i].execution);
printf("Deadline (for EDF): ");
scanf("%d", &t[i].deadline);
printf("Period (for RMS): ");
scanf("%d", &t[i].period);
}
edf(t, n);
rms(t, n);
return 0;
}
```

O/P:

```
Enter number of tasks: 3

Task 1
Burst Time: 3
Deadline (for EDF): 10
Period (for RMS): 6

Task 2
Burst Time: 2
Deadline (for EDF): 5
Period (for RMS): 4
```

Task 3

Burst Time: 1

Deadline (for EDA): 8

Period (for RMS): 3

---- Earliest Deadline First (EDA) scheduling ----

ID	BT	Deadline	CT	WT	TAT
1	3	10	6	3	6
2	2	5	2	0	2
3	1	8	3	2	3

---- Rate monotonic scheduling (RMS) ----

ID	BT	Period	CT	WT	TAT
1	3	6	6	3	6
2	2	4	3	1	3
3	1	3	1	0	1

Gronth Chart (EDF):

T_2	T_2	T_3	T_1	T_1	T_1
0	1	2	3	4	5	6

31/3/26

Gronth Chart: (RMS):

T_3	T_2	T_2	T_1	T_1	T_1
0	1	2	3	4	5	6

3) The following program consists of 3 concurrent process and 3 binary semaphores. The semaphores are initialized as S0=1, S1=0, S2=0. find out how many times process P0 will print "0". Asume the order of execution as P0, P1, P2, P0, P1

```
Process P0:                 Process P1:          Process P2:
while (true) {                  wait (S1);           wait (S2);
    wait (S0);                  signal (S0);         signal (S0);
    Printf ("0");
    signal (S1);
    signal (S2);
```

Ans:

Initial	$S_0 = 1$	$S_1 = 0$	$S_2 = 0$	Remark
P_0	0	1	1	0
P_1	1	0	1	—
P_2	1	0	0	—
P_0	0	1	1	0 0
P_1	1	0	1	—

P_0 will print 0 two times.

2) consider two processes A and B. Two semaphore variables S and T are used to synchronize the process. S is initialized to 0 and T is initialized to 1. Processes are scheduled in the following order:
A B A B B A A B A

what will be printed on the screen?

```
Process A:                  Process B:
while (1)                   while (1) {
{                               wait (T);
    wait (S);                   Print 'I';
    Print 'P';                 print 'S';
    Print 'P';                 signal (S);
    signal (T);             }
}
```

Initial	S=0	S=1	Remarks
A	0	1	Process A is blocked (wait)
B	1	0	||
A	0	1	|| PP
B	1	0	|| PP ||
B	1	0	Process B is blocked
A	0	1	|| PP || PP
A	0	1	|| PP Process A is blocked
B	1	0	|| PP || PP ||
A	0	1	|| PP || PP || PP

8) Producer - consumer problem using semaphores

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#define N 5
int buffer[N], in=0, out=0, item=0, n;
sem_t empty, full, mutex;
void *producer(void *arg) {
for (int i=0; i<n; i++) {
sem_wait (&empty);
sem_wait (&mutex);
buffer[in] = item;
Printf("Producer: %d at buffer[%d]\n", item, in);
in = (in+1) %N;
item ++;
sem_post (&mutex);
sem_post (&full);
}
return NULL;
}
void *consumer (void *arg) {
for (int i=0; i<n; i++) {
sem_wait (&full);
sem_wait (&mutex);
int var = buffer[out];
Printf("consumed: %d from buffer[%d]\n", var, out);
out = (out+1) %N;
sem_post (&mutex);
sem_post (&empty);
}return NULL;
}
int main() {
pthread_t P, C;
Printf("Enter number of items:");
```

```
scanf("%d", &n);

sem_init(&semMutex, 0, 1);
sem_init(&full, 0, 0);
sem_init(&empty, 0, n);

pthread_create(&p, NULL, producer, NULL);
pthread_create(&c, NULL, consumer, NULL);

pthread_join(p, NULL);
pthread_join(c, NULL);

return 0;
}
```

O/P:

Enter number of items: 5

Produced: 0 at buffer[0]
Produced: 1 at buffer[1]
Consumed: 0 from buffer[0]
Produced: 2 at buffer[2]
Consumed: 1 from buffer[1]
Produced: 3 at buffer[3]
Consumed: 2 from buffer[2]
Produced: 4 at buffer[4]
Consumed: 3 from buffer[3]
Consumed: 4 from buffer[4]

21/4/26

2) Dining Philosopher's problem:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <stdlib.h>

int N;
sem_t *forks, mutex;
void *philosopher (void*num) {
int i = *(int*) num;
for (int k=0; k<2; k++) {
Printf("Philosopher %d is thinking \n", i);
sem_wait(&mutex);
sem_wait (&forks[i]);
Printf("Philosopher %d picked up left fork %d. \n", i, i);
sem_wait(&forks[(i+1)%N]);
Printf("Philospher %d picked up right fork %d \n", i, (i+1)%N);
sem_post(&mutex);
Printf("Philospher %d is eating \n", i);
sem_post(&forks[i]);
sem_post(&forks[(i+1)%N]);
Printf("Philosopher %d put down forks %d and %d \n", i, i, (i+1)%N);
}
return null;
}
int main() {
Printf("Enter number of Philosophers:");
Scanf("%d", &N);
Pthread_t P[N];
int id[N];
forks = (sem_t*) malloc(N*sizeof(sem_t));
sem_init(&mutex, 0, 1);
for (int i=0; i<N; i++)
sem_init(&forks[i], 0, 1);
for(int i=0; i<N; i++) {
```

```
id[i] = i;
Pthread_create(&P[i], NULL, Philosopher, &id[i]);
}
for(int i=0; i<N; i++)
Pthread_join(P[i], NULL);
return 0;
}
```

OP:

```
Enter number of philosophers: 4
Philosopher 1 is thinking
Philosopher 1 picked up left fork 1
Philosopher 1 picked up right fork 2
Philosopher 3 is thinking
Philosopher 3 picked up left fork 3
Philosopher 0 ~~picked~~ is thinking
Philosopher 3 picked up ~~left~~ right fork 0
Philosopher 1 is eating
Philosopher 3 is eating
Philosopher 1 put down forks 1 and 2
Philosopher 0 picked up left fork 0
Philosopher 1 is thinking
Philosopher 0 picked up right fork 1
Philosopher 0 is eating
Philosopher 0 put down forks 0 and 1
Philosopher 0 is thinking
Philosopher 1 picked up left fork 1
Philosopher 1 picked up right fork 2
Philosopher 2 is thinking
Philosopher 3 put down forks 3 and 0
Philosopher 3 is thinking
Philosopher 0 picked up left fork 0
Philosopher 1 is eating
```

Philosopher 1 put down forks 1 and 2
Philosopher 0 picked up right fork 1
Philosopher 0 put down forks 0 and 1.
Philosopher 0 ~~put down to~~ is eating.
Philosopher 2 picked up left fork 2
Philosopher 2 picked up right fork 3
Philosopher 2 is eating
Philosopher 2 is eating
Philosopher 2 put down forks 2 and 3
Philosopher 2 is thinking
Philosopher 3 picked up left fork 3
Philosopher 3 picked up right fork 0
Philosopher 3 is eating
Philosopher 3 put down forks 3 and 0
Philosopher 2 picked up left fork 2
Philosopher 2 picked up right fork 3
Philosopher 2 is eating
Philosopher 2 put down forks 2 and 3.

Safety Algorithm: (→ deadlock)

```
#include<stdio.h>
#include<stdbool.h>
int main() {
int P, R;
int i, j;
Printf("Enter number of Processes:");
scanf("%d", &P);
printf("Enter number of resource types:");
scanf("%d", &R);
int Allocation[P][R], max[P][R], Need[P][R];
int Available[R], work[R];
bool finish[P];
int safesequence[P];
Printf("\nEnter Allocation matrix:\n");
for(i=0; i<P; i++) {
for(j=0; j<R; i++) {
scanf("%d", &Allocation matrix:\n");
for(i=0; i<P; i++) {
for(j=0, j<R; j++) {
scanf("%d", &Allocation[i][j]);
}
}
Printf("\nEnter max matrix:\n");
for(i=0, i<P; i++) {
for(j=0; j<R; j++) {
scanf("%d", &max[i][j]);
}
}
```

```
printf("\n enter Available Resources:\n");
for (j=0; j<R; j++) {
scanf("%d", &Available[j]);
}

for (i=0; i<P; i++){
for(j=0; j<R; j++) {
need[i][j] = max[i][j] - Allocation[i][j];
}
}
for(j=0; j<R; j++) {
work[j] = Available[j];
}

for(i=0; i<P; i++) {
finish[i] = false;
}
int count=0;
while(count<P){
bool found = false;
for(i=0; i<P; i++) {
if(finish[i] == false) {
bool canExecute = true;
for(j=0; j<R; j++) {
if (need[i][j] > work[j]) {
canExecute = false;
break;
}
}
if(canExecute) {
for(j=0; j<R; j++) {
work[j] += Allocation[i][j];
}
finish[i] = true;
safeSequence[count++] = i;
found = true;
```

```
    }
  }
}
if (!found == false) {
  break;
}
}
bool safe = true;
for(i=0; i<p; i++){
if (finish[i] == false) {
 safe = false;
  break;
  }
}
if (safe) {
printf("\n system is in safe state\n safe sequence:");
for (i=0; i<p; i++) {
printf("%d ", safe sequence[i]);
}
printf("\n");
} else {
printf("\n system is in unsafe state (Potential deadlock)\n");
}
return 0;
}
```

O/P:

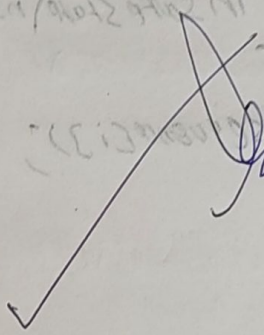
Enter number of processes: 5
Enter number of resource types: 4

Enter Allocation matrix:

0 0 1 2
1 0 0 0
1 3 5 4
0 6 3 2
0 0 1 4

Enter max matrix:

0 0 1 2
1 7 5 0
2 3 5 6
0 6 5 2
0 6 5 6



Enter Available Resources:

1 5 2 0

System is in safe state.

Bankers Algorithm:

```
#include <stdio.h>
#include <stdbool.h>
int main() {
    int P, R, i, j;
    Printf("Enter number of processes:");
    Scanf("%d", &P);
    Printf("Enter number of resource types:");
    Scanf("%d", &R);
    Printf("Enter number
    int Allocation[P][R], max[P][R], need[P][R];
    int Available[R], work[R];
    bool Finish[P];
    int safesequence[P];
    int Request[R], Process;
    Printf("\n Enter Allocation matrix:\n");
    for (i=0; i<P; i++)
      for (j=0; j<R; j++)
        Scanf("%d", &Allocation[i][j]);
    Printf("enter max matrix:\n");
    for (i=0; i<P; i++)
      for (j=0; j<R; j++)
        Scanf("%d", &max[i][j]);
    Printf("\n Enter Available Resources:\n");
    for (j=0; j<R; j++)
    Scanf("%d", &Available[j]);

for (i=0; i<P; i++)
for (j=0; j<R; j++)
    need[i][j]=max[i][j]-Allocation[i][j];

Printf("\n Enter Process number (0 to %d):", P-1);
Scanf("%d", &Process);

Printf("Enter Request Vector:\n");
for (j=0; j<R; j++)
    Scanf("%d", &Request[j]);

for (j=0; j<R; j++) {
if (Request[j]>need[Process][j]) {
Printf("\n Error: Request exceeds need:\n");
```

```
return 0;
}
if (Request[i] > Available[i]) {
    Printf("\nResources not available. Process must wait.\n");
    return 0;
  }
}
for(i=0; j<R; j++) {
Available[j] -= Request[j];
Allocation[Process][j] += Request[j];
Need[Process][j] -= Request[j];
}
for(j=0; j<R; j++)
  work[j] = Available[j];
for(i=0; i<P; j++)
  finish[i] = false;
int count=0;
while count<P) {
  bool found = false;
  for(i=0; j<P; j++) {
  if(!finish[i]) {
  bool canExecute= true;
  for(j=0; j<R; j++) {
  if(Need[i][j] > work[j]) {
  canExecute = false;
  break;
  }
}
if(canExecute) {
  for(j=0; j<R; j++)
    work[j] += Allocation[i][j];
finish[i] = true;
safeSequence[count++]=i;
found = true;
  }
}
}
```

```
if(!found)
  noseq!
}
if (count==n){
printf("\n Request GRANTED. System is in SAFE STATE\n sequence: ");
for(i=0; i<n; i++)
   printf("P%d ", safeSequence[i]);
} else {
printf("P%d ", safeSequence[i]);
printf("\n Request DENIED. System would be UNSAFE\n");
}
 return 0;
}
```

O/P:

Enter number of processes: 5

Enter number of resource types: 3

Enter Allocation matrix:

```
0  1  0
2  0  0
3  0  2
2  1  1
0  0  2
```

Enter max matrix:

```
7  5  3
3  2  2
9  0  2
2  2  2
4  3  3
```

Enter Avail Resources:

3 3 2

Enter process number (0 to 4): 1

Enter request vectors:

1 0 2

Request GRANTED. System is in SAFE STATE

Sequence: P_1 P_3 P_4 P_0 P_2

Deadlock detection:

```
#include <stdio.h>
#include <stdbool.h>
int main() {
int P, R, i, j;
printf("enter number of processes: ");
scanf("%d", &P);
printf("enter number of resource types: ");
scanf("%d", &R);
int Allocation[P][R], Request[P][R];
int Available[R], work[R];
bool finish[P];
printf("\n enter Allocation matrix: \n");
for (i=0; i<P; i++)
  for (j=0; j<R; j++)
    scanf("%d", &Allocation[i][j]);
printf("\n enter Request matrix: \n");
for(i=0; i<P; i++)
for(j=0; j<R; j++)
scanf("%d", &Request[i][j]);
printf("\n enter Available Resources: \n");
for(j=0; j<R; j++)
scanf("%d", &Available[j]);
for(j=0; j<R; j++)
work[j] = Available[j];
for(i=0; i<P; i++) {
finish[i] = false;
for(j=0; j<R; j++) {
if(Allocation[i][j] != 0) {
finish[i] = false;
break;
}
}
}
bool found;
do {
```

```
found = false;
for (i=0; i<p; i++) {
if (!finish[i]) {
bool canbeexecute = true;
for (j=0; j<R; j++) {
if (Request[i][j] > work[i]) {
canbeexecute = false;
break;
}
}
if (canbeexecute) {
for (j=0; j<R; j++)
work[i] += allocation[i][j];
finish[i] = true;
found = true;
}
}
}
} while (found);

bool deadlock = false;
for (i=0; i<p; i++) {
if (!finish[i]) {
deadlock = true;
printf("process Pid is deadlocked\n", i);
}
}
if(!deadlock)
printf("\nno Deadlock Detected\n");
return 0;
}
```

O/P:

Enter number of Processes : 3
Enter number of resource types: 2

Enter Allocation matrix:

1	0
0	1
1	1

Enter Request matrix:

1	1
1	0
0	1

Enter Available Resources:

Process P_0 is deadlocked
Process P_1 is deadlocked
Process P_2 is deadlocked

5/5/26

Memory Allocation: Best, Worst, First fit

```
#include <stdio.h>
#define MAX 100
void firstfit(int blocks[], int m, int processes[], int n)
{
    int allocation[MAX];
    for(int i=0; i<n; i++)
        allocation[i] = -1;
    for(int i=0; i<n; i++)
    {
        for(int j=0; j<m; j++)
        {
            if(blocks[j] >= processes[i])
            {
                allocation[i] = j;
                blocks[j] -= processes[i];
                break;
            }
        }
    }
    printf("\nFirst fit Allocation\n");
    printf("Process\tsize\tBlock\n");
    for(int i=0; i<n; i++)
    {
        printf("%d\t%d\t", i+1, processes[i]);
        if(allocation[i] != -1)
            printf("%d\n", allocation[i]+1);
        else
            printf("Not Allocated\n");
    }
}
void bestfit(int blocks[], int m, int processes[], int n)
{
    int allocation[MAX];
    for(int i=0; i<n; i++)
        allocation[i] = -1;
    for(int i=0; i<n; i++)
    {
        int bestIdx = -1;
        for(int j=0; j<m; j++)
        {
```

```
if (blocks[j] >= processes[i])
{
    if (bestIdx == -1 || blocks[j] < blocks[bestIdx])
        bestIdx = j;
    }
}
if (bestIdx != -1)
{
    allocation[i] = bestIdx;
    blocks[bestIdx] -= processes[i];
    }
}
Printf("\n Best fit Allocation \n");
Printf("Process \t size \t block \n");
for(int i=0; i<n; i++)
{
    Printf("%d \t %d \t", i+1, processes[i]);
    if(allocation[i] != -1)
        Printf("%d \n", allocation[i]+1);
    else
        Printf("Not Allocated \n");
    }
}
void worstFit(int block[], int m, int processes[], int n)
{
    int allocation[MAX];
    for(int i=0; i<n; i++)
        allocation[i] = -1;
    for(int i=0; i<n; i++)
    {
        int worstIdx = -1;
        for(int j=0; j<m; j++)
        {
            if blocks[j] >= processes[i])
            {
                if (worstIdx == -1 || blocks[j] > blocks[worstIdx])
                    worstIdx = j;
                }
            }
```

```
if (worst idx != -1)
{
  allocation[i] = worstIdx;
  blocks[worstIdx] = processes[i];
  }
}
printf("\nWorst Fit Allocation\n");
printf("Process\tSize\tBlock\n");

for(int i=0; i<n; i++)
{
  printf("%d\t%d\t", i+1, processes[i]);
  if (allocation[i] != -1)
      printf("%d\n", allocation[i]+1);
  else
      printf("Not Allocated\n");
  }
}
int main()
{
  int m, n, choice;
  int blocks[MAX], processes[MAX];
  int tempBlocks[MAX];
printf("Enter number of memory blocks:\n");
for(int i=0; i<m; i++)
   scanf("%d", &blocks[i]);
printf("enter number of processes:");
scanf("%d", &n);
printf("enter sizes of processes:\n");
for(int i=0; i<n; i++)
   scanf("%d", &processes[i]);
do
 {
   printf("\n---Memory Allocation menu---:\n");
   printf("1. FirstFit\n");
   printf("2. BestFit\n");
   printf("3. WorstFit\n");
   printf("4. Exit\n");
   printf("Enter your choice:");
```

```
scanf("%d", &choice);
for (int i=0; i<m; i++)
    tempBlocks[i] = blocks[i];
switch (choice)
{
    case 1:
        firstFit(tempBlocks, m, processes, n);
        break;
    case 2:
        bestFit(tempBlocks, m, processes, n);
        break;
    case 3:
        worstFit(tempBlocks, m, processes, n);
        break;
    case 4:
        printf("Exiting Program....\n");
        break;
    default: 1
        printf("Invalid Choice!\n");
}
} while (choice != 4);
return 0;
}
```

O/P:

Enter number of memory blocks: 5
Enter sizes of memory blocks:
400 700 200 300 600
Enter no. of processes = 4
Enter sizes of processes: 212 512 312 526.

First fit Allocation

process no	process size	Block no
1	212	1
2	512	2
3	312	5
4	526	Not Allocated

for all algorithms

Process no	Process size	Block no
1	212	4
2	512	5
3	312	1
4	526	2

worst fit allocation

Process no	Process size	Block no
1	212	2
2	512	5
3	312	1
4	526	not allocated

12/5/26

Placement algorithm.

```
#include <stdio.h>
#define max 50.
int isPresent (int frames[], int n, int page) {
    for (int i=0; i<n; i++) {
        if (frames[i] == page)
            return 1;
    } return 0;
}
void Fifo (int pages[], int n, int capacity) {
int frames[max], index = 0, faults = 0;
for (int i=0; i< capacity; i++) { frames[i] = -1; }
Printf("\n. Fifo Page Replacement\n");
for (int i=0; i<n; i++) {
if (!isPresent (frames, capacity, pages[i])) {
    frames[index] = pages[i];
    index = (index+1) % capacity;
    faults ++; }
Printf("Page %d →", pages[i]);
for (int j=0; j< capacity; j++) {
  if (frames[j] != -1)
    printf ("%d", frames[j]);
else
    printf ("- ")
}
printf("\n");
. printf("Total Page faults = %d\n", faults);
}
void LRU (int pages[], int n, int capacity)
{
  int frames[max], time[max];
  int faults = 0, counter = 0;
  for (int i=0; i< capacity; i++)
  {
    frames[i] = -1;
    time[i] = 0;
  }
Printf("\n LRU Page Replacement\n");
```

```
for (int i=0; i<n; i++)
{
int found = 0;
for (int j=0; j<capacity; j++)
{
if (frames[j]==pages[i])
{
counter++;
time[j]=counter;
found=1;
break;
}
}
if (!found)
{
int pos=0;
for (int j=1; j<capacity; j++)
{
if (time[j]<time[pos])
pos=j;
}
counter++;
frames[pos]=pages[i];
time[pos]=counter;
faults++;
}
printf("Page %d->", pages[i]);
for(int j=0; j<capacity; j++)
{
if (frames[j]!=-1)
printf("%d", frames[j]);
else
printf("- ");
}
printf("\n");
}
printf("Total Page faults = %d\n", faults);
}
void optimal(int pages[], int n, int capacity)
{
int frames[MAX];
int faults=0;
```

```
for (int i=0; i<capacity; i++)
    frames[i]=-1;
printf("\n Optimal Page Replacement\n");
for(int i=0; i<n; i++)
{
if(!isPresent(frames, capacity, pages[i]))
{
int pos=-1, farthest=i+1;
for(int j=0; j<capacity; j++)
{
int k;
for(k=i+1; k<n; k++)
{
if(frames[j]==pages[k])
{
if(k>farthest)
{
farthest=k;
pos=j;
}
break;
}
}
if(k==n)
{
pos=j;
break;
}
}
if(pos==-1)
pos=0;
frames[pos]=pages[i];
faults++;
}
printf("Page %d->", pages[i]);
for(int j=0; j<capacity; j++)
{
if(frames[j]!=-1)
printf("%d", frames[j]);
else
printf("-");
```

```
    }
    printf("\n");
}
printf("Total Page faults = %d\n", faults);
}
int main()
{
int pages[MAX], n, capacity;
printf("Enter number of pages: ");
scanf("%d", &n);
printf("Enter page reference string:\n");
for(int i=0; i<n; i++)
scanf("%d", &pages[i]);
printf("Enter frame capacity: ");
scanf("%d", &capacity);
FIFO(pages, n, capacity);
LRU(pages, n, capacity);
optimal(pages, n, capacity);
return 0;
}
```

O/P:

Enter number of pages: 12
Enter page reference string:
1 2 3 4 1 2 5 1 2 3 4 5

Enter frame capacity: 3

Fifo Page Replacement:

Page 1 -> 1 - -
Page 2 -> 1 2 -
Page 3 -> 1 2 3
Page 4 -> 4 2 3
Page 1 -> 4 1 3
Page 2 -> 4 1 2
Page 5 -> 5 1 2
Page 1 -> 5 1 2
Page 2 -> 5 1 2
Page 3 -> 5 3 2
Page 4 -> 5 3 4
Page 5 -> 5 3 4
Total Page faults = 9

LRU Page Replacement:

Page 1 -> 1 - -
Page 2 -> 1 2 -
Page 3 -> 1 2 3
Page 4 -> 4 2 3
Page 1 -> 4 1 3
Page 2 -> 4 1 2
Page 5 -> 5 1 2
Page 1 -> 5 1 2
Page 2 -> 5 1 2
Page 3 -> 3 1 2
Page 4 -> 3 4 2
Page 5 -> 3 4 5
Total Page Faults = 10

Optimal Page Replacement:

Page 1 -> 1 - -
Page 2 -> 1 2 -
Page 3 -> 1 2 3
Page 4 -> 1 2 4
Page 1 -> 1 2 4
Page 2 -> 1 2 4
Page 5 -> 1 2 5
Page 1 -> 1 2 5
Page 2 -> 1 2 5
Page 3 -> 3 2 5
Page 4 -> 4 2 5
Page 5 -> 4 2 5
Total Page faults = 7